

**Міністерство освіти і науки
України Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій**

Звіт
про виконання лабораторної роботи №1
“Організація професійного Python-проєкту та Git”

Виконав:
студент 3 курсу
групи ФЕП-33с
Качмар Денис
Викладач:
Проф. Свелеба Сергій Андрійович

Львів - 2026

Хід роботи

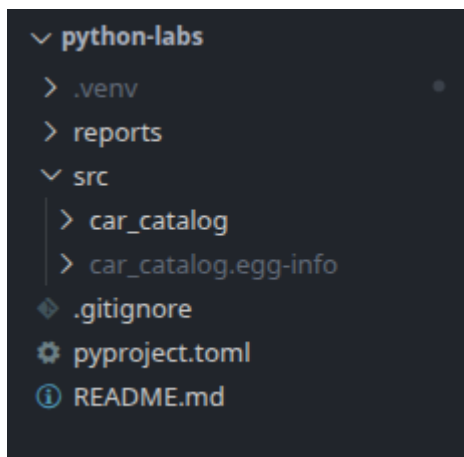
Мета: Ознайомитися з основними принципами організації професійного Python-проєкту, навчитися створювати ізольоване середовище розробки, формувати структуру проєкту за принципом src-layout, працювати з модулями та пакетами Python, створювати файл `pyproject.toml`, використовувати Git для контролю версій та оформлювати базову документацію проєкту.

Варіант: 5 (Каталог автомобілів)

Постановка задачі: Створити структурований Python-проєкт з використанням віртуального середовища (`.venv`), застосувавши підхід `src-layout`. Розділити логіку програми "Каталог автомобілів" на окремі модулі (моделі, сервіси, точка входу). Налаштувати конфігураційні файли проєкту, додати документацію та зберегти історію розробки за допомогою системи контролю версій Git щонайменше у 3 логічні коміти, після чого завантажити репозиторій на GitHub.

Структура проєкту:

Проєкт організовано за принципом `src-layout`:



Опис створених modules:

Програму розділено на такі модулі:

- `__init__.py`: перетворює каталог `car_catalog` на Python-пакет та містить версію.
- `models.py`: містить структуру даних (dataclass) `Car`, яка описує сутність автомобіля (марка, модель, рік, ціна, пробір).

- services.py: реалізує бізнес-логіку застосунку — функції для пошуку, фільтрації та обчислень середньої ціни авто в каталозі.
- main.py: точка входу (entry point) застосунку, де ініціалізуються демонстраційні дані та запускається інтерактивне CLI-меню для користувача.

Текст основних фрагментів коду:

Фрагмент src/car_catalog/models.py:

```
from dataclasses import dataclass

@dataclass
class Car:
    brand: str
    model: str
    year: int
    price: float
    mileage: float

    @property
    def full_name(self) -> str:
        return f"{self.brand} {self.model}"
```

Фрагмент src/car_catalog/services.py:

```
from car_catalog.models import Car

def search_by_brand(cars: list[Car], brand: str) -> list[Car]:
    return [c for c in cars if c.brand.lower() == brand.lower()]

def filter_by_year(cars: list[Car], year: int) -> list[Car]:
    return [c for c in cars if c.year == year]

def find_most_expensive_car(cars: list[Car]) -> Car | None:
    if not cars:
        return None
    return max(cars, key=lambda c: c.price)

def calculate_average_price(cars: list[Car]) -> float:
    if not cars:
        return 0.0
    return sum(c.price for c in cars) / len(cars)

def find_lowest_mileage_car(cars: list[Car]) -> Car | None:
    if not cars:
        return None
```

```
return min(cars, key=lambda c: c.mileage)
```

Фрагмент src/car_catalog/main.py:

```
def main() -> None: run_menu(create_demo_cars())  
if name == "main": main()
```

Пояснення pyproject.toml:

```
[build-system]
```

```
requires = ["setuptools>=70"]
```

```
build-backend = "setuptools.build_meta"
```

```
[project]
```

```
name = "car-catalog"
```

```
version = "0.1.0"
```

```
description = "Laboratory project for Professional Python"
```

```
requires-python = ">=3.11"
```

```
[project.scripts]
```

```
car-catalog = "car_catalog.main:main"
```

Секція [build-system] вказує, які інструменти використовуються для збірки проєкту (у цьому випадку setuptools). Секція [project] містить метадані (назву, версію, опис, вимоги до версії Python). Секція [project.scripts] створює консольну команду car-catalog для зручного запуску програми з терміналу.

Вміст .gitignore:

```
.venv/  
__pycache__/  
*.pyc  
.pytest_cache/  
.idea/
```

```
.vscode/  
dist/  
build/  
*.egg-info/  
.env
```

Фрагмент README:

Python Labs

Author

Навчальний проєкт з дисципліни «Професійний Python».

Містить каталог автомобілів (Варіант 5), який буде розвиватися протягом курсу.

Author

Студент: Качмар Денис Богданович

Група: ФЕП-21

Requirements

Python 3.11+

Installation & Run

```
```bash
```

```
python3 -m venv .venv
```

```
source .venv/bin/activate
```

```
python3 -m pip install -e .
```

```
car-catalog
```

### Screenshots запуску application:

```
@kaory car-catalog

--- Каталог автомобілів ---
1. Показати всі авто
2. Пошук за маркою
3. Фільтр за роком
4. Найдорожче авто
5. Середня ціна
6. Найменший пробіг
7. Вихід
Оберіть команду: 1
Audi A6 C5 | 2003 | 4500.00$ | 350000.0 км
Toyota Camry | 2021 | 28000.00$ | 45000.0 км
BMW X5 | 2019 | 45000.00$ | 120000.0 км
Ford Focus | 2015 | 10000.00$ | 150000.0 км

--- Каталог автомобілів ---
1. Показати всі авто
2. Пошук за маркою
3. Фільтр за роком
4. Найдорожче авто
5. Середня ціна
6. Найменший пробіг
7. Вихід
```

Результат git status:

```
@kaory git status
On branch main
Your branch is based on 'origin/main', but the upstream is gone.
(use "git branch --unset-upstream" to fixup)

nothing to commit, working tree clean
(.venv)
```

Результат git log –oneline:

```
6d1cc78 Create project structure
...skipping...
de639d3 (HEAD -> main) Add application entry point and documentation
347bfa5 Add car model and business logic
6d1cc78 Create project structure
```

Відповіді на контрольні запитання:

1. **Що таке Python-проект?** Це структурований набір файлів, каталогів, модулів, залежностей та конфігурацій, об'єднаних для вирішення певного завдання.
2. **Чим проект відрізняється від одного Python script?** Скрипт зазвичай складається з одного файла, що швидко стає незручним при збільшенні коду, тоді як проект розділений на логічні компоненти з налаштованою системою збірки та тестами.
3. **Що таке Python module?** Це звичайний файл із розширенням .py, який містить код.

4. **Що таке Python package?** Це каталог, що містить файли модулів і спеціальний файл `__init__.py`.
5. **Для чого потрібний `__init__.py`?** Щоб вказати інтерпретатору Python, що даний каталог є пакетом, вміст якого можна імпортувати.
6. **Що таке `src-layout`?** Це структура проєкту, де весь програмний код (модулі, пакети) зберігається у вкладеному каталозі `src/`.
7. **Які переваги має `src-layout`?** Це дозволяє чітко розмежувати конфігураційні файли (в корені) від самого коду, запобігає помилкам випадкового імпорту та покращує надійність тестування.
8. **Для чого використовується `virtual environment`?** Для ізоляції встановлених бібліотек та залежностей проєкту, щоб вони не конфліктували з іншими проєктами або глобальною системою.
9. **Як створити `.venv`?** За допомогою команди `python -m venv .venv`.
10. **Як активувати `virtual environment`?** У Linux/macOS командою `source .venv/bin/activate`, у Windows — `.venv\Scripts\activate`.
11. **Для чого використовується `pip`?** Це стандартний менеджер пакунків Python, який дозволяє встановлювати, оновлювати та видаляти зовнішні бібліотеки.
12. **Чому рекомендується `python -m pip`?** Це гарантує, що інструмент встановлення `pip` буде запущено саме з того середовища Python, яке зараз активоване (наприклад, з віртуального).
13. **Для чого потрібний `pyproject.toml`?** Файл конфігурації, який містить метадані та правила збірки сучасного Python-проєкту.
14. **Що містить секція `[project]`?** Основні відомості про пакет: назву, версію, короткий опис, мінімальну підтримувану версію Python.
15. **Що таке `editable installation`?** Режим встановлення через `pip install -e .`, який дозволяє редагувати сирцевий код проєкту, при цьому зміни автоматично застосовуються без необхідності перевстановлення пакета.
16. **Для чого використовується `if __name__ == "__main__":`?** Щоб функція (наприклад, `main()`) викликалась тільки за умови прямого запуску файлу, і не викликалась, якщо файл імпортується як модуль в іншому місці.
17. **Що таке `Git`?** Розподілена система контролю версій (DVCS) для збереження історії змін, повернення до попередніх станів та роботи в команді.
18. **Що таке `repository`?** Сховище проєкту під управлінням `Git`, яке містить усі файли, історію змін та конфігурацію системи контролю версій.
19. **Що таке `commit`?** Це збережений стан файлів (знімок) у репозиторії на певний момент часу із прикріпленням текстовим описом змін.

20. **Для чого використовується git status?** Для перегляду поточного стану відстежуваних і невідстежуваних файлів та виявлення внесених змін у робочому каталозі.
21. **Для чого git add?** Для додавання змінених файлів до індексу (staging area) з метою підготовки їх до створення коміту.
22. **Для чого git commit?** Для фіксації проіндексованих змін та створення нового запису в історії репозиторію.
23. **Що показує git log?** Хронологічну історію створених комітів у репозиторії.
24. **Для чого потрібен .gitignore?** Щоб перелічити файли або папки (наприклад, кеш, лог-файли), які Git має ігнорувати і не додавати в репозиторій.
25. **Чому .venv не потрібно комітити?** Тому що віртуальне середовище містить дуже багато файлів, залежить від операційної системи розробника і його легко можна відтворити з нуля командою створення.
26. **Для чого потрібний README?** Це головний документаційний файл, який описує суть проєкту та містить інструкції з його налаштування і запуску.
27. **Що take type hints?** Це механізм анотування типів у Python (наприклад, `def main() -> None`), який підказує розробнику та редактору коду очікувані типи даних аргументів і результатів виконання функцій.
28. **Для чого програму розділяють на modules?** Це дозволяє логічно структурувати програму: відокремити введення/виведення від моделей даних та бізнес-логіки, що полегшує тестування і підтримку коду.
29. **Що take business logic?** Це та частина програмного коду, яка виконує основні обчислення, правила обробки та перетворення даних.
30. **Чому потрібно відокремлювати application entry point від business logic?** Це дозволяє імпортувати бізнес-функції для тестування або в інші частини проєкту, не запускаючи при цьому інтерфейс самої програми.

**Висновок:** Під час виконання лабораторної роботи було створено повноцінний структурований Python-проєкт згідно з сучасними стандартами розробки. Набуто практичних навичок роботи з віртуальними середовищами (venv), організації коду за шаблоном src-layout та розділення логіки на окремі незалежні модулі. Засвоєно принципи налаштування проєктів через `pyproject.toml`, а також закріплено навички роботи з системою контролю версій Git: створення комітів для логічних етапів розробки, написання `.gitignore` та збереження коду на віддаленому репозиторії GitHub.